Technical Documentation

RAG-Based Healthcare Assistant using LangGraph & HITL

Introduction

Retrieval-Augmented Generation (RAG) combines Information Retrieval and Large Language Models (LLMs) to generate grounded and accurate responses. Instead of answering only from pretrained knowledge, the system retrieves relevant information from a healthcare PDF knowledge base and uses it to generate responses. RAG helps reduce hallucinations, improves factual accuracy, and enables domain-specific healthcare assistance.

System Architecture

The system contains the following components:

PDF Knowledge Base → Text Chunking → Embedding Generation → Chroma Vector Database → Retriever → LangGraph Workflow → LLM → HITL Feedback Loop

Workflow:

User Query → Conversational Memory → Retriever → Relevant Chunks → LLM → Generated Response → Human Review → Regeneration (if needed)

Design Decisions

- **Chunk Size:** 500 with 100 overlap to balance retrieval accuracy and context continuity.
- **Embedding Model:** all-MiniLM-L6-v2 for lightweight and fast semantic retrieval.
- **Vector Database:** ChromaDB for efficient local vector search.
- **Prompt Design:** Structured prompts force grounded healthcare responses and reduce hallucinations.

LangGraph Workflow

LangGraph manages:

- Query processing
- Retrieval flow
- Conversational memory
- HITL refinement loop
- State management

Graph State:

- query
- context
- answer
- feedback

- chat_history
- retry_count

Main Node:

Chat Node

- Reads query and conversation history
- Retrieves relevant chunks
- Generates/refines answers
- Updates conversation memory

Conditional Logic

The system supports:

- Healthcare information queries
- Conversational follow-up questions
- Human feedback refinement

Routing:

- If feedback = “yes” → End workflow
- If feedback = “no” → Refine answer
- If retry limit exceeded → Stop refinement gracefully

HITL (Human-in-the-Loop)

The HITL workflow allows users to review AI responses and request improvements.

HITL Flow:

AI Response → User Feedback → Answer Refinement → Repeat until satisfied

Benefits:

- Improves answer quality
- Reduces hallucinations
- Enhances conversational experience
- Builds user trust

Limitations:

- Additional response latency
- Retrieval quality impacts refinement quality
- Multiple retries may still fail

Challenges & Trade-offs

- Small embedding models improve speed but reduce semantic understanding.
- Small chunks improve precision but may lose contextual completeness.
- Larger LLMs improve reasoning but increase inference cost.

Testing Strategy

The project was tested using:

- Basic healthcare queries
- Conversational follow-up questions
- HITL refinement testing
- Unknown query testing

Sample Queries:

- What is diabetes?
- medicines for it
- explain more clearly
- What is machine learning?

Expected unknown-query behavior:
“I don't have enough medical information in the knowledge base.”

Future Enhancements

- Multi-document support
- Better feedback categorization
- Long-term conversational memory
- Query rewriting for follow-up questions
- Streamlit/FastAPI deployment
- Docker and cloud deployment

Conclusion

This project demonstrates a conversational RAG-based healthcare assistant using LangGraph and HITL. The system combines vector retrieval, conversational memory, and iterative human feedback refinement to generate grounded healthcare responses while reducing hallucinations and improving user interaction.

Workflow Diagram

START

↓

User Query

↓

Chat Node

↓

Retrieve Relevant Chunks

↓

Generate Answer using LLM

↓

Show Answer to User

↓

Human Review (Helpful?)

↓

Yes | No

Yes → END

No → Regenerate Answer

↓

Retry Limit Check

↓

Retry | Stop

Retry → Generate Answer

Stop → END